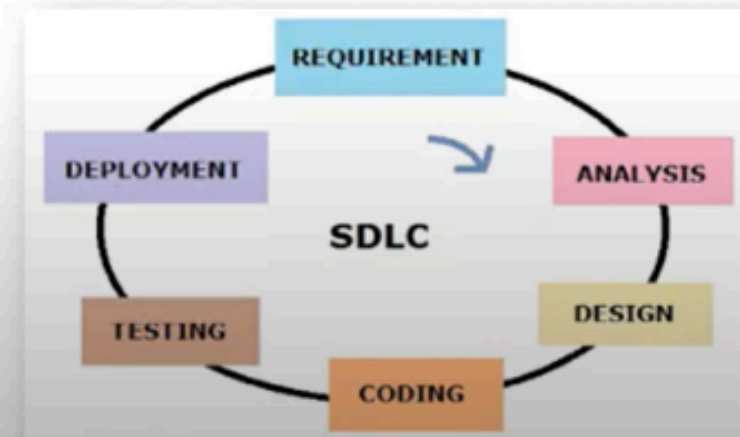


# Software Design Process

- Software design is a process to transform user requirements into some suitable form, which helps the programmer in software coding and implementation.
- Software design is the first step in SDLC, which moves the focus from problem domain to solution domain.
- It tries to specify how to fulfill the requirements mentioned in SRS.
- Done by Software Design Engineer OR UI/UX Designer.



## Objectives of Software Design

1. **Correctness:** Software design should be correct as per requirement.
2. **Completeness:** The design should have all components like data structures, modules, and external interfaces, etc.
3. **Efficiency:** Resources should be used efficiently by the program.
4. **Flexibility:** Able to modify on changing needs.
5. **Consistency:** There should not be any inconsistency in the design.
6. **Maintainability:** The design should be so simple so that it can be easily maintainable by other designers.

# Software Design Process

- The software design process can be divided into the following 3 levels of phases of design:
  1. **Interface Design:** It focused on interaction between the system and the users devices. Scenario & Behavioral based diagram used. Not focus on internal structure.
  2. **Architectural Design:** It focused on major components of a system their responsibilities, properties, interfaces, relationships and interactions between them. Class based & flow based diagram used.
  3. **Detailed Design:** It focused on internal elements of all major system components, their properties, relationships, processing , algorithms and data structures.

# Software Quality Guidelines

1. A design is generated using the recognizable architectural styles and compose a good design characteristic of components and it is implemented in evolutionary manner for testing.
2. A design of the software must be modular i.e the software must be logically partitioned into sub elements.
3. In design, the representation of data , architecture, interface and components should be distinct.
4. A design must carry appropriate data structure and recognizable data patterns.
5. Design components must show the independent functional characteristic.
6. A design creates an interface that reduce the complexity of connections between the components with external environment.
7. A design must be derived using the repeatable method.
8. The notations should be use in design which can effectively communicates its meaning.

# Software Quality Attributes

The attributes of design name as 'FURPS' are as follows:

1. **Functionality:** It evaluates the feature set and capabilities of the program.
2. **Usability:** It is accessed by considering the factors such as human factor, consistency and documentation.
3. **Reliability:** It is evaluated by measuring parameters like frequency and security of failure, output result accuracy, recovery from failure and program predictability.

**4. Performance:** It is measured by considering processing speed, response time, resource consumption, throughput and efficiency.

**5. Supportability:** It combines the ability to extend the program, adaptability, serviceability, Testability, compatibility and configurability are the terms using which a system can be easily installed and found the problem easily.

Supportability also consists of more attributes such as modularity, reusability, robustness, security, portability, scalability.



# Software Design Concepts

- 1. Abstraction**
- 2. Architecture**
- 3. Design Patterns**
- 4. Modularity**
- 5. Information Hiding**
- 6. Functional Independence**
- 7. Refinement**
- 8. Refactoring**
- 9. Object-Oriented Design Concept**

# Abstraction

- Abstraction is used to hide background details or unnecessary implementation about the data.
- So that users see only required information.

## Type 1: Procedural Abstraction:

- There is collections of subprograms.
- One is hidden group another is visible group of functionalities. **Example:**



### **Private:**

Fuel\_machine()  
Set\_top\_speed()  
Develop\_engine()

### **Public:**

Turn\_on() , Turn\_off()  
Accelerate(), Break()

## Type 2: Data Abstraction:

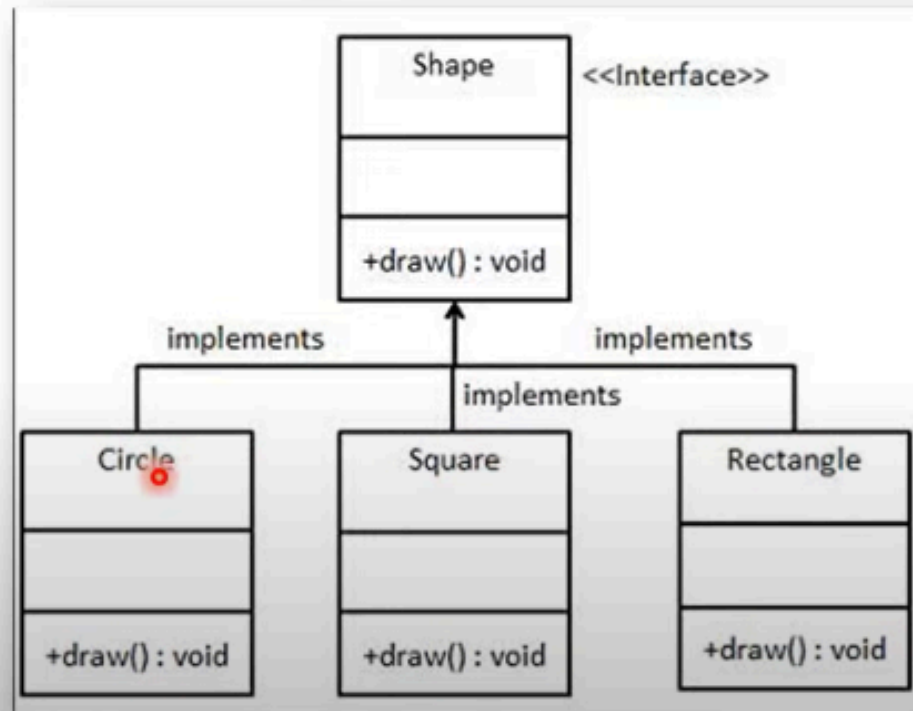
- Collections of data that describe data objects.
- Show representation data & hide manipulation data.
- **Examples:** Data Structure Programs directly used Push(), Pop(), Top() and Empty() methods.

# Architecture

- The architecture is the structure of program modules where they interact with each other in a specialized way.
- **Structural Properties:** Architectural design represent different types of components, modules, objects & relationship between these.
- **Extra-Functional Properties:** How design architecture achieve requirements of Performance, Capacity, Reliability, Security, Adaptability & other System Characteristics.
- **Families of related systems:** The architectural design should draw repeatable patterns. They have ability to reuse repeatable blocks.

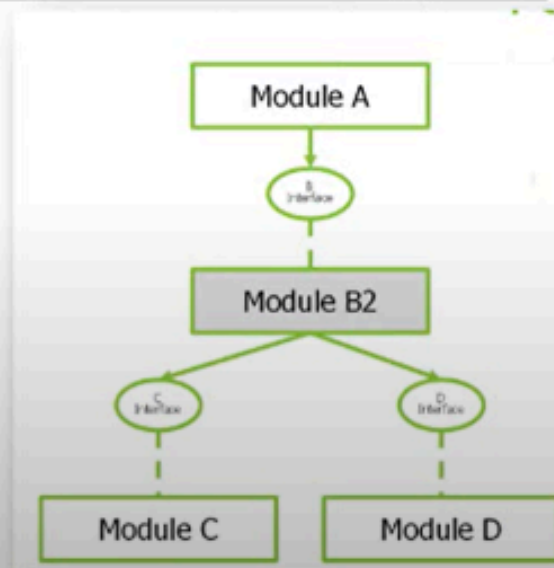
# Design Patterns

- The pattern simply means a repeated form or design in which the same shape is repeated several times to form a pattern.
- **Example:**



# Modularity

- Modularity simply means dividing the system or project into smaller parts to reduce the complexity of the system or project.
- After developing the modules, they are integrated together to meet the software requirements.
- Modularizing a design helps to effective development, accommodate changes easily, conduct testing, debugging efficiently and conduct maintenance work easily.



# Information Hiding

- Modules should be specified and designed in such a way that the data structures and algorithm details of one module are not accessible to other modules.
- They pass only that much information to each other, which is required to accomplish the software functions.
- The way of hiding unnecessary details in modules is referred to as information hiding.

The image displays a web interface with two forms side-by-side on a dark blue background. The 'Sign Up' form on the left includes fields for First Name, Last Name, Email, Password, and Confirm Password, each with a green 'Submit' button at the bottom. The 'Login' form on the right includes fields for Email and Password, with a green 'Submit' button. Both forms have links for users who are not new (e.g., 'Not have an account? Sign Up here' and 'Already have an account? Login here').

**Sign Up**  
It's free and only takes a minute

First Name

Last Name

Email

Password

Confirm Password

[Submit](#)

By clicking the Sign Up button you agree to our [Terms and Conditions](#) and [Privacy Policy](#)

[Already have an account? Login here](#)

**Login**

Email

Password

[Submit](#)

[Not have an account? Sign Up here](#)

# Functional Independence

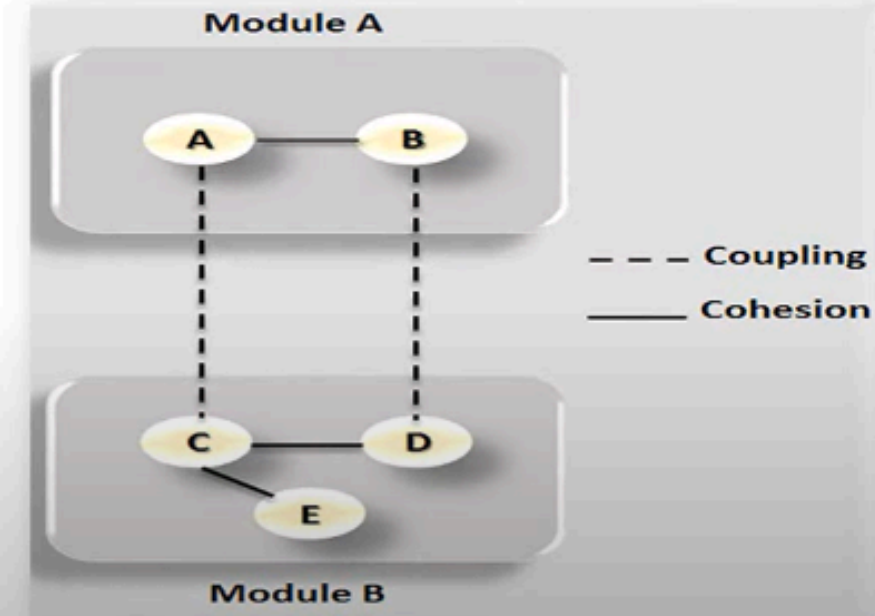
- The functional independence is the concept of separation and related to the concept of modularity, abstraction and information hiding.

## Criteria 1: Coupling

- The degree in which module is “connected” to other module in the system.
- Low Coupling necessary in good software.

## Criteria 2: Cohesion

- The degree in which module perform functions in inner module in the system.
- High Cohesion necessary in good software.



# Refinement

- Refinement is a top-down design approach.
- It is a process of elaboration.
- A program is established for refining levels of procedural details.
- A hierarchy is established by decomposing a statement of function in a stepwise manner till the programming language statement are reached.

## Example:

INPUT

Get number 1 (Integer)

Get number 2 (Integer)

PROCESS

OUTPUT

INPUT

Get number 1 (Integer)

Get number 2 (Integer)

While (Invalid Number)

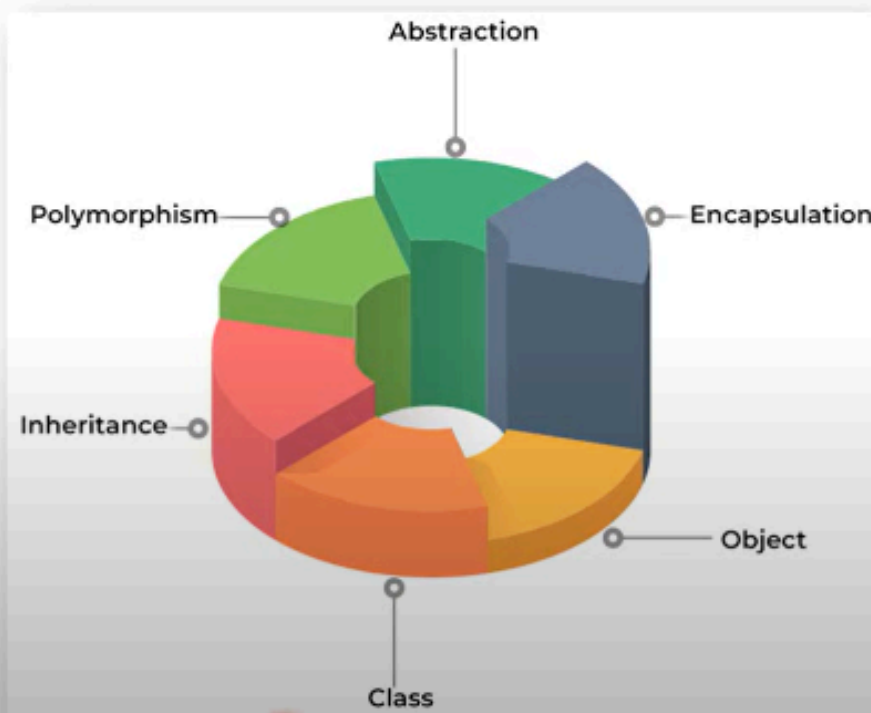
EXIT

# Refactoring

- Refactoring is the process of changing the internal software system in a way that it does not change the external behavior of the code still improves its internal structure.
- When software is refactored the existing design is examined for redundancy, unused design elements, unnecessary design algorithms, poorly constructed data, inappropriate data structure or any other design failure that can be corrected for better design.

# Object Oriented Design Concepts

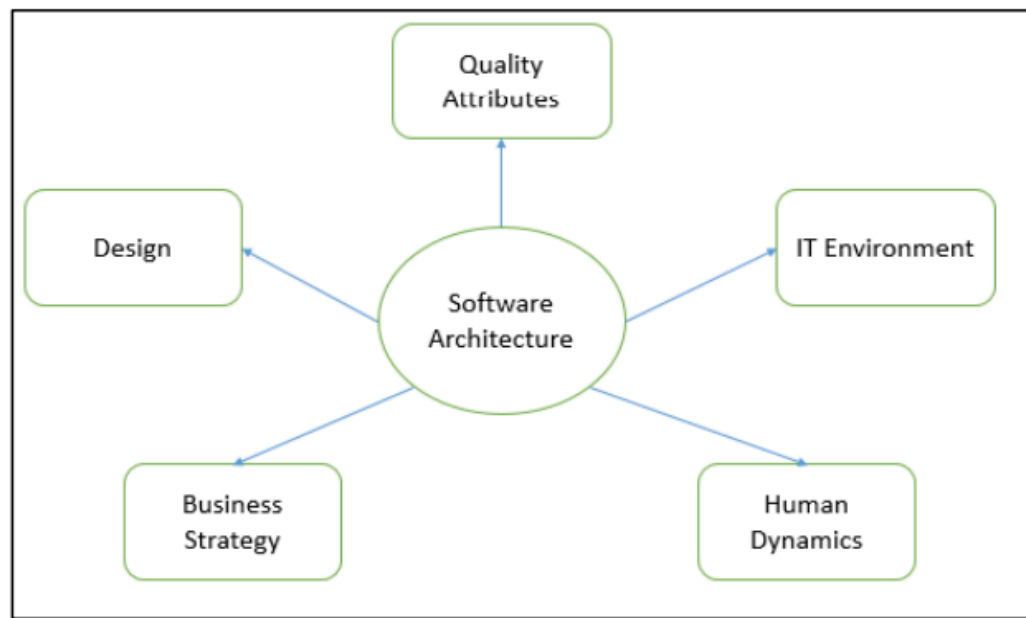
- Object Oriented is a popular design approach for analyzing and designing an application.
- Advantage is that faster, low cost development and creates a high quality software.



# Software Architecture

Architecture serves as a **blueprint for a system**. It provides an abstraction to manage the system complexity and establish a communication and coordination mechanism among components.

- It defines a **structured solution** to meet all the technical and operational requirements, while optimizing the common quality attributes like performance and security.
- Further, it involves a set of significant decisions about the organization related to software development and each of these decisions can have a considerable impact on quality, maintainability, performance, and the overall success of the final product. These decisions comprise of –
  - Selection of structural elements and their interfaces by which the system is composed.
  - Behavior as specified in collaborations among those elements.
  - Composition of these structural and behavioral elements into large subsystem.
  - Architectural decisions align with business objectives.
  - Architectural styles guide the organization.



The architecture of a system describes its major components, their relationships (structures), and how they interact with each other. Software architecture and design is a process that includes several contributory factors such as Business strategy, quality attributes, human dynamics, design, and IT environment.

We can segregate Software Architecture and Design into two distinct phases: Software Architecture and Software Design. In **Architecture**, nonfunctional decisions are cast and separated by the functional requirements. In **Design**, functional requirements are accomplished.

# Types of Architecture

---

There are four types of architecture from the viewpoint of an enterprise and collectively, these architectures are referred to as **enterprise architecture**.

- **Business architecture:** Defines the strategy of business, governance, organization, and key business processes within an enterprise and focuses on the analysis and design of business processes.
- **Application (software) architecture:** Serves as the blueprint for individual application systems, their interactions, and their relationships to the business processes of the organization
- **Information architecture:** Defines the logical and physical data assets and data management resources.
- **Information technology (IT) architecture:** Defines the hardware and software building blocks that make up the overall information system of the organization.

# Importance of Software Architecture

1. **Security:** The system is secured against malicious users by encryption or any other security measures due to layered software architecture.
2. **Performance:** It handle request and response of the page in minimum time.
3. **Maintainability:** Architectural design process uses easily modifiable and replaceable components which is easy to change them over time according to the new requirements.
4. **Safety:** Avoid critical functionalities in small components & improve communication of the system.
5. **Availability:** Architectural design process includes corresponding components, functionalities for handling the occurrence of any type of errors.

# Data Design

Translates data objects as part of the analysis model into data structures at the software component level and, when necessary, a database architecture at the application level.

## 1. Data design at the Architectural Level:

- The challenge for a business has been to extract useful information from this data environment, particularly when the information desired is cross functional.

## 2. Data design at the Component Level:

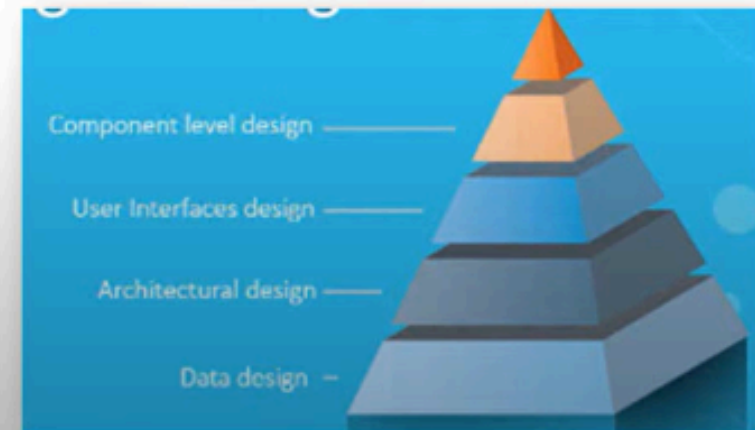
- focuses on the representation of data structures that are directly accessed by one or more software components.
- The following set of principles for data specification
  1. The systematic analysis principles applied to function and behavior should also be applied to data.
  2. All the data structures and the operations to be performed on each should be identified.
  3. A data dictionary should be established and used to define both data and program design
  4. Low-level data design decisions should be deferred until late in the design process.
  5. The representation of data structure should be known only to those modules that must make direct use of the data contained within the structure.
  6. A library of useful data structures and the operations that may be applied to them should be developed
  7. A software design and programming language should support the specification and realization of abstract data types.

# Design Models

- The design phase of software development, transforming the customer requirements as described in the SRS documents into design forms.
- Designing a model is an important phase and is a multi-process that represent the data structure, program structure, interface characteristic and procedural details.

➤ **There are four types of design elements / models**

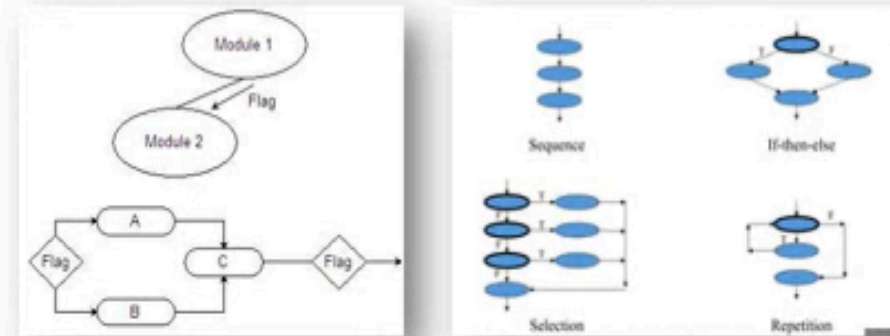
1. Data Design Element / Model
2. User Interface Design Element / Model
3. Architectural Design Element / Model
4. Component Level Design Element / Model



# Component Level Design Model

## ➤ What is Component?

- A component is a modular, portable, replaceable and reusable set of well-defined functionalities.



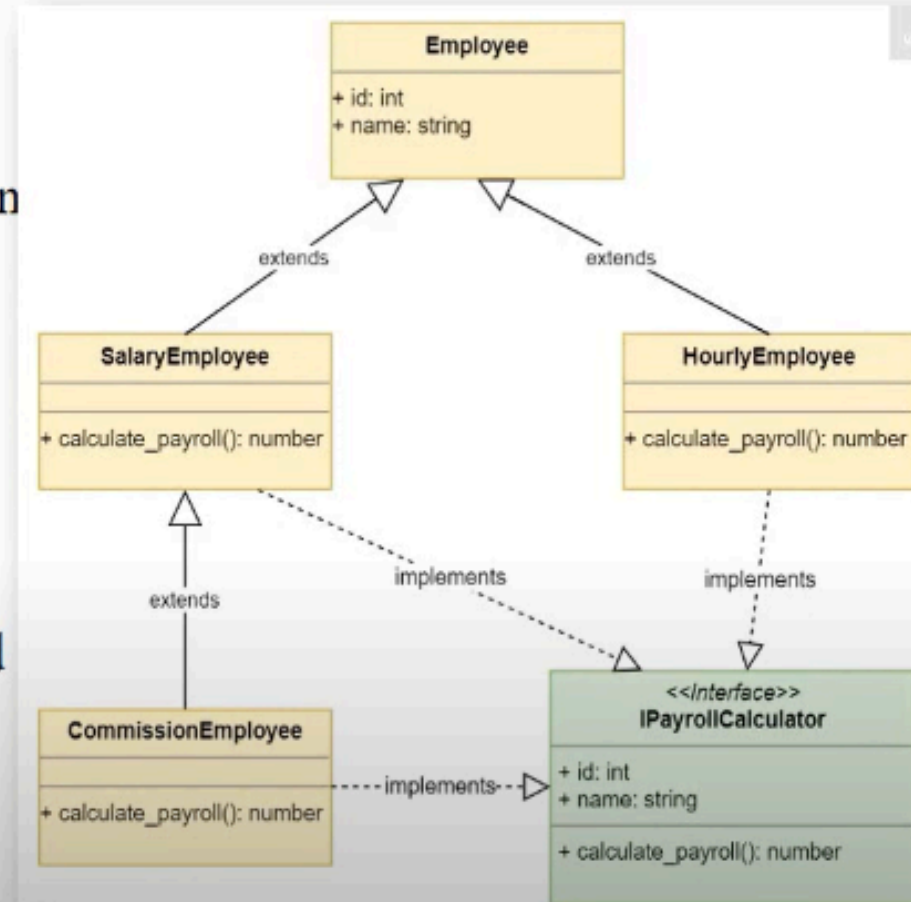
## ➤ What is Component Level Design?

- The component-based architecture focuses on breaking the software design into small individual modules.
- The component design describes communication, interfaces, algorithms & functionalities of each component regarding the whole software design.
- Component Level Design notations are Activity Diagram, Data Flow Diagram, Conditional Notations & Tabular forms.

# Component Design View

## 1. Object Oriented View:

- An object-oriented view is a set of collaborating classes in module mentioned in architectural design
- It view all the objects, methods, properties and functionalities mentioned in classes.
- It also view how classes in each module are connected with each other.
- It detailed out data structure & algorithms required for processing.



# Architectural Design Model

- Architecture Design Model serves as a blueprint for a system.
- The architecture focuses on the early design decisions.

## Architectural Design Styles describe:

- Set of hardware & software components that will perform a function required by the system.  
Eg. Database, Modules, Frameworks etc.
- Set of connectors will help in coordination & communication between the components.
- Conditions that how components can be integrated to form the system.
- Semantic models that help the designer to understand the overall properties of the system.

# Decisions of Architectural Design

- The architectural design process differs as the system differs depending upon the type of system being developed.



➤ **There are some common decisions that should be taken care of in any design process.**

- ☐ How can the system be distributed across the network?
- ☐ Which approach can be used to structure the system?
- ☐ Which architectural styles are suitable for the proposed system?
- ☐ How can software architecture be documented?
- ☐ How can the system be decomposed into modules?
- ☐ What control strategy must be used to control the operation of the components in the system?
- ☐ How can architectural design be analyzed?

# Taxonomy of Architectural styles

➤ Architectural styles is establish a complete structure & components of all over the system.

1. Data Centered Architecture
2. Data Flow Architecture
3. Object Oriented Architecture
4. Layered Architecture



# Data Centered Architecture

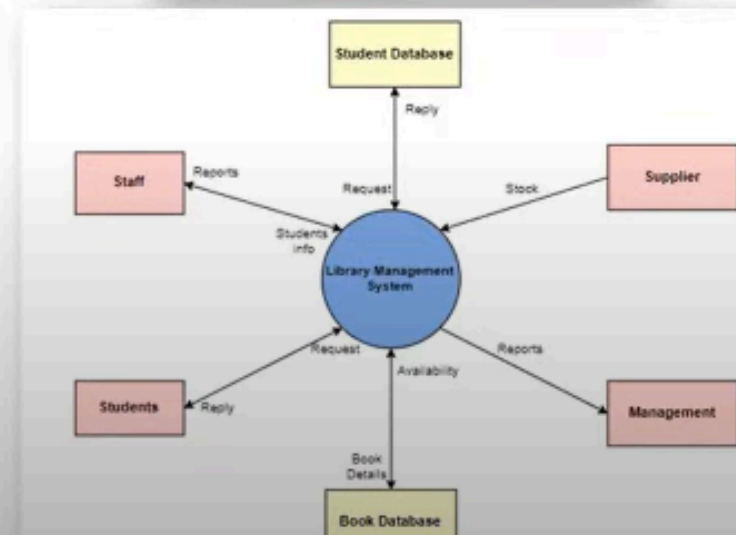
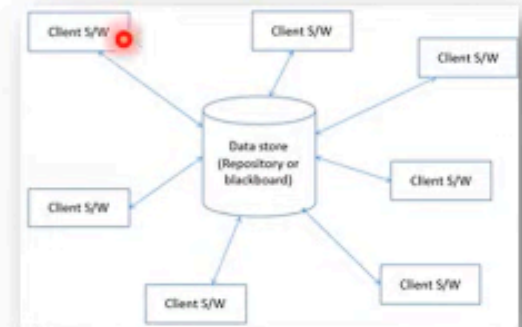
- Data store at the center of this architecture and is accessed frequently by everyone.
- Update, add, delete or modify the data present within the store.
- It is widely used in DBMS, Library Information System etc.

## Advantages:

1. Repository of data is independent of clients
2. It may be simple to add additional clients.
3. Modification can be very easy.

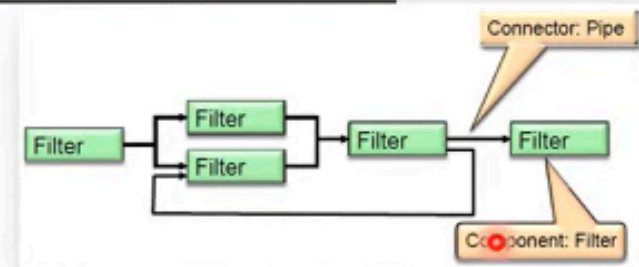
## Disadvantages:

1. Data replication or duplication is possible.
2. Changes in data structure highly affect the clients.



# Data Flow Architecture

- This architecture is used when input data to be transformed into output data through a series of computational manipulative components.
- **Pipe** is a connector which passes the data directionally from one filter to the next.
- **Filter** is a component reads the data from its input pipes and performs its function.
- This data and places the result on all output pipes.

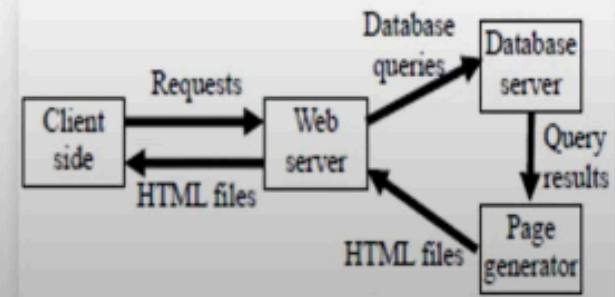


## Advantages:

1. With this design, concurrent execution is supported.

## Disadvantages:

1. Does not allow greater user engagement.



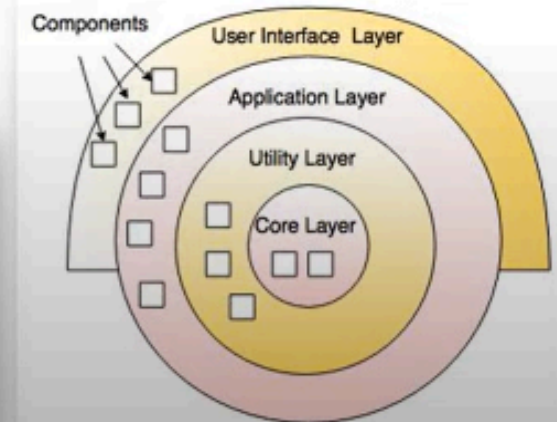
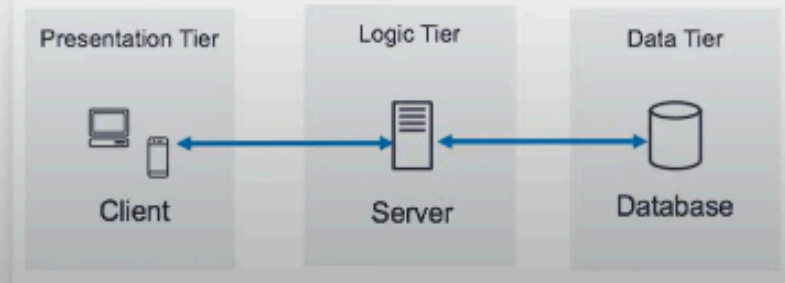
# Object Oriented Architecture

- Objects are the foundational building blocks for all kinds of software applications.
1. **Object:** Object is an instance of a class. Example Student S, Person;
  2. **Class:** It defines all attributes, methods, which represents the functionality of the object.
  3. **Encapsulation:** It is the process of binding similar types of elements of an abstraction.
  4. **Abstraction:** It is the removal of irrelevant essentials from users.
  5. **Inheritance:** It deriving a new class from existing one. It increases code reusability.
  6. **Polymorphism:** It has multiple forms. Ex: draw graphic objects circle, rectangle, triangle
  7. **Message Passing:** Sending and receiving data among objects through function parameters.



# Layered Architecture

- Data moves from one level to another level for processing is called layered architecture.
- Number of different layers are defined every layer performing well-defined set of operations.
- **Outer layer** components manage the user interface operations.
- **Inner layer** components will perform the operating system interfacing.
- **Intermediate layers** provide utility services and application software functions.
- **Example:** E-commerce web applications development like Amazon.



# Architectural Styles And Patterns

- The Software That Is Built For Computer-based Systems Also Exhibits One Of Many Architectural Styles.
- Each Style Describes A System Category That Encompasses

1. A Set Of *Components*

2. Set Of *Connectors*

3. *Constraints*

4. *Semantic Models*

An ***architectural pattern***, imposes a transformation the design of architecture. However, a pattern differs from a style in a number of fundamental ways:

The scope of a pattern is less broad, focusing on

- one aspect of the architecture rather than the architecture in its entirety.
- A pattern imposes a rule on the architecture, describing how the software will handle some aspect of its functionality at the infrastructure level.
- Architectural patterns tend to address specific behavioral issues within the context of the architectural.

# Architectural Design Model

- Architecture Design Model serves as a blueprint for a system.
- The architecture focuses on the early design decisions.

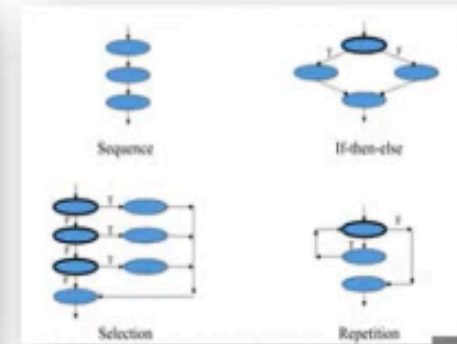
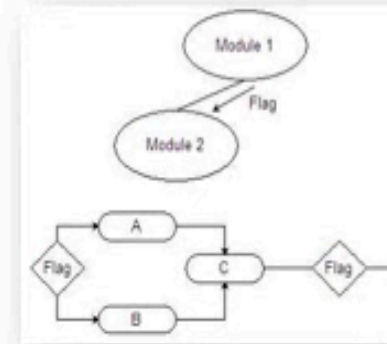
## Architectural Design Styles describe:

- Set of hardware & software components that will perform a function required by the system.  
Eg. Database, Modules, Frameworks etc.
- Set of connectors will help in coordination & communication between the components.
- Conditions that how components can be integrated to form the system.
- Semantic models that help the designer to understand the overall properties of the system.

# Component Level Design Model

## ➤ What is Component?

- A component is a modular, portable, replaceable and reusable set of well-defined functionalities.



## ➤ What is Component Level Design?

- The component-based architecture focuses on breaking the software design into small individual modules.
- The component design describes communication, interfaces, algorithms & functionalities of each component regarding the whole software design.
- Component Level Design notations are Activity Diagram, Data Flow Diagram, Conditional Notations & Tabular forms.

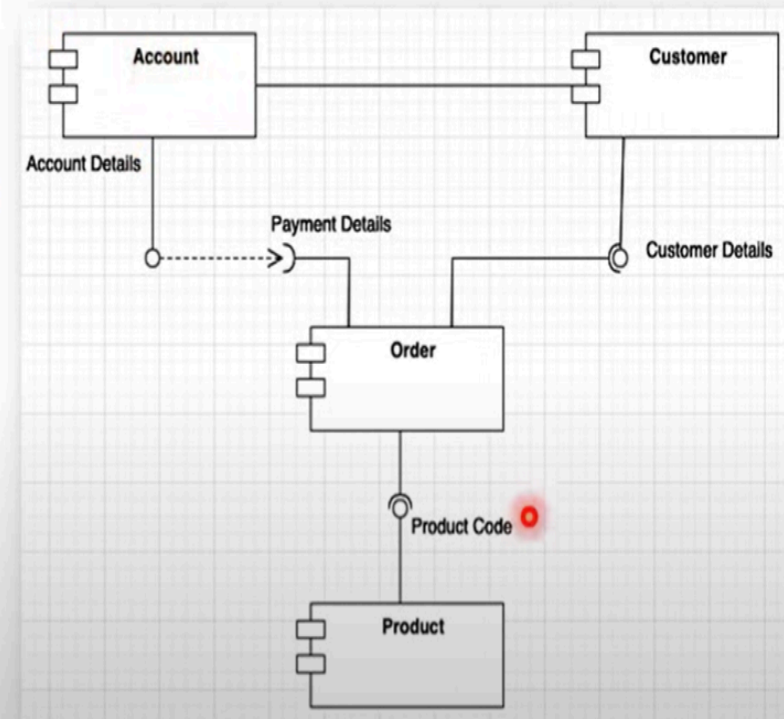
## Steps of Component Level Design

1. Identify all design classes that corresponds to problem domain.
2. Identify all design classes that corresponds to infrastructure domain. **Ex.** GUI Class, Database management class, OS Communication class etc.
3. Elaborate all design classes that are not acquired as reusable component. **Ex.** Coupling & Cohesion methods like message details between classes, data type, data structure & data flow between classes.
4. Describe persistent data source & identify classes require to manage them. **Ex.** Classes which data from databases.
5. Develop behavioral representation for a component & class. **Ex.** Classes, Operations & there Interactions through Activity & State diagram.
6. Elaborate deployment diagram. **Ex.** Hardware, OS, Server of components.
7. Factors every component level design representations. **Ex.** Diff. design solutions that provide good characteristics.

## Component Design Example

- It has Account, Customer, Order & Product this Four Components.
- Every object is an individual component but is related to each other at the same time.

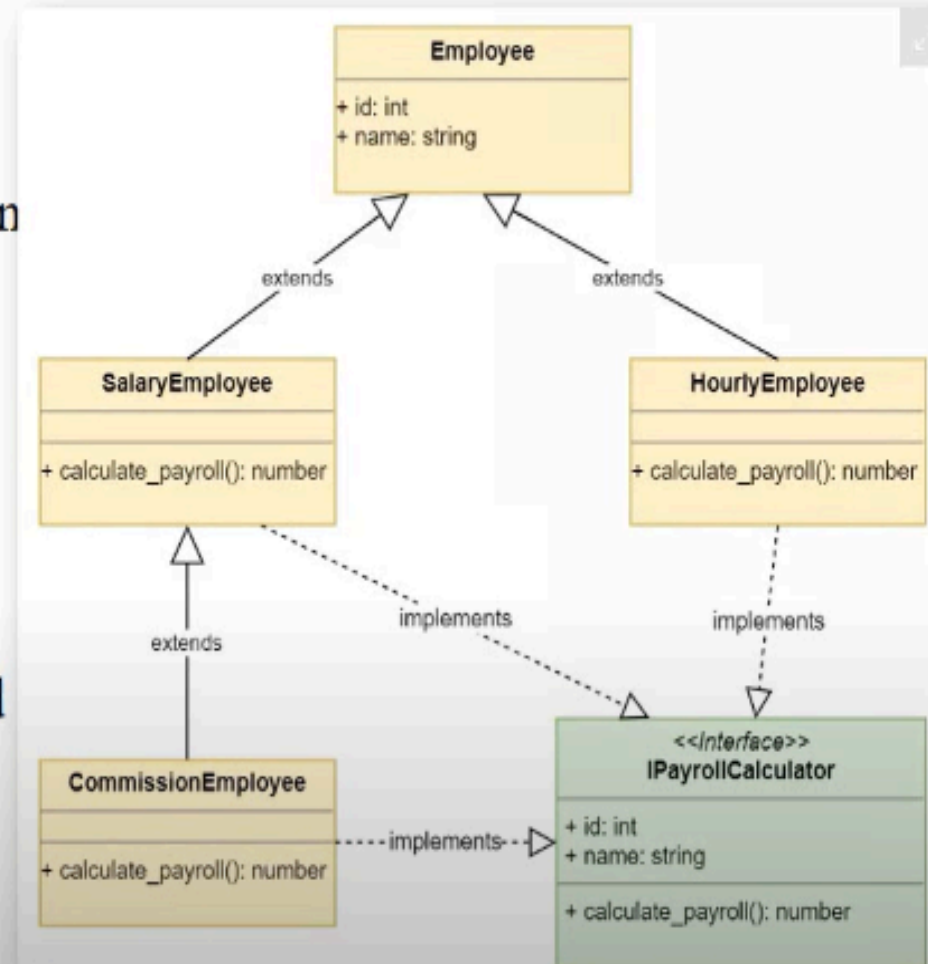
1. Customer will go online site and search product.
2. By selecting product, the customer will place an order.
3. System will store customer & his account information in Order Database.
4. The order would be generated against the customer and stored in the Database.



# Component Design View

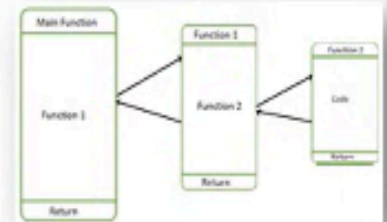
## 1. Object Oriented View:

- An object-oriented view is a set of collaborating classes in module mentioned in architectural design
- It view all the objects, methods, properties and functionalities mentioned in classes.
- It also view how classes in each module are connected with each other.
- It detailed out data structure & algorithms required for processing.



## 2. Conventional View:

- It is viewed functional element of a program that integrates the all logical & internal operations performed in class.
- It also view calling and coordinating the request like calling functions, parameter passing, data passing between the of other modules.
- They analyze complete view of data flow model & state diagram.



## 3. Process-related View:

- The process-related view majorly focuses on the reusability feature of the component design.
- Databases and libraries are used to store pre-existed modules.
- This pre-existed data used for creating new modules.

# Component Level Design Guidelines

## 1. Components:

- Naming conventions should be established for components that are specified as part of the architectural model which is understandable to all stakeholders. Ex. Calculator
- Use Infrastructure component name that mentioned implementation details. Ex. Stack

## 2. Interface:

- Recognizes and discover these independent entities as new components.
- Interfaces provide important information about communication and collaboration.

## 3. Dependencies and Inheritance:

- Models any dependencies from left to right and Inheritance from top (base class) to bottom (derived classes).
- Used complete concept of Coupling & Cohesion of system.

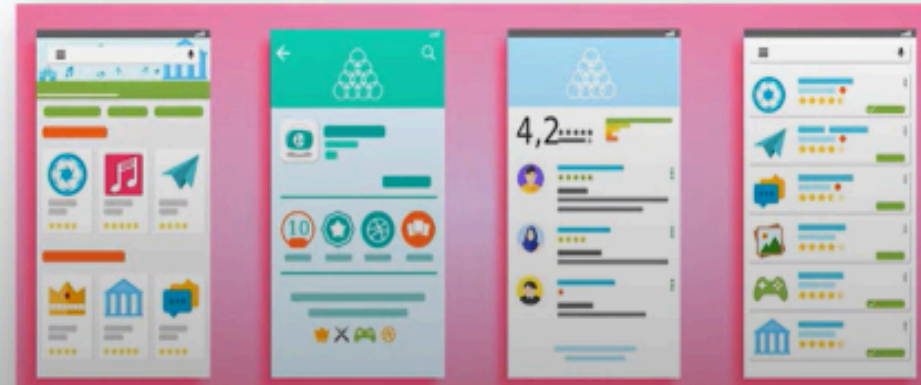
# User Interface Design Model

- User interface is the front-end application view to which user interacts with the software.
- It determines how commands are given to the computer or the program and how data is displayed on the screen.



➤ **The software becomes more popular if its user interface is:**

1. Attractive
2. Simple to use
3. Responsive in short time
4. Clear to understand
5. Consistent on all interfacing screens



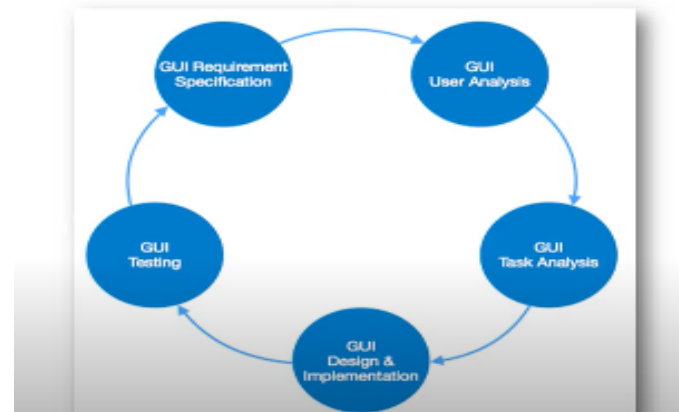


## Type 2: Graphical User Interface

- Graphical User Interface provides the simple interactive interface to interact with the system.
- GUI can be a combination of both hardware and software.
- Graphical user interfaces are easy to learn as compared to the command-line interface.
- GUI provides multiple windows to the user simultaneously to interact with the system.



# User Interface Design Process



## User Interface Design Process

1. **GUI Requirement Gathering:** Designer analyze all functional & non functional requirements mentioned & discussed with customers.
2. **User Analysis:** Designer studies who is going to use the software GUI. The target audience matters as the design details change according to the knowledge level of the user.
3. **Task Analysis:** Designer analyze tasks, sub tasks & flow of the system.
4. **GUI Design & Implementation:** Designer generate actual GUI design by using different implementation tools like Wavemaker, Visual Studio etc.
5. **Testing:** GUI Testing done by Inner designer team, Organization & different stockholders in project.

## User Interface Design Principles

1. **User Familiarity:** The interface should be based of user oriented terms & concepts.

**Example:** In Windows OS, Terms like Desktop, Document, Folder, Rename, Power etc.

2. **Consistency:** The system command and menus should have the same format & parameter.

**Example:** Group of fonts, sizes, colors etc.

3. **Minimal Surprise:** Users never like to see the system working in an unexpected manner.

They get frustrated in such cases. Develop as per the user tendency.

**Example:** Group of similar type of features , In MS Words, File -> New, Open, Save, Print,

Close , Restart system

**4. Recoverability:** Users most often make mistakes while working with the system. These mistakes can be reduced but cannot be completely eliminated.

**Example:** Undo, Rename

**5. User Diversity:** The interface should provide appropriate interaction facilities for the various types of the system user.

**Example:** Larger Font, Privacy Settings etc.

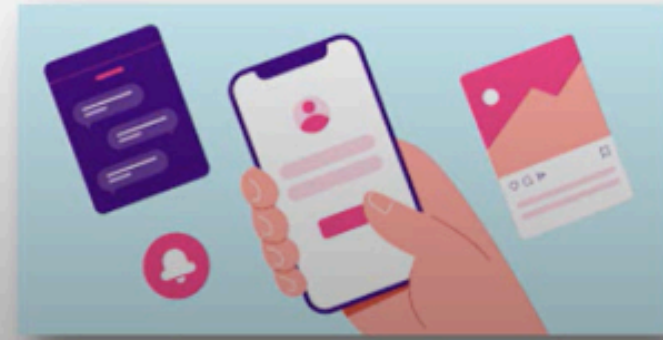
**6. User Guidance:** The user interface should provide meaningful feedback when errors occur and provide different levels of help and advice.

**Example:** User Manual, Forgot Password etc.

# Design Golden Rules

## 1. Place the user in control.

- ✓ The user should be able to easily enter and exit the mode.
- ✓ Provide flexible interaction using keyboard, mouse or touch screen etc.
- ✓ User must be able to interrupt the sequence to do some other work.
- ✓ Display descriptive messages and text.
- ✓ Allow users to customize the interface like short cut keys etc.
- ✓ Hide technical internal details from users.
- ✓ Allow users to directly manipulate interface objects.



## 2. Reduce Users' Memory Load

- ✓ Reduce demand on short-term memory.
- ✓ New features updated in system.
- ✓ Define shortcuts that are intuitive.
- ✓ The visual layout of the interface should be based on a real-world metaphor.
- ✓ Disclose information in a hierarchical & progressive fashion.

## 3. Make the Interface Consistent

- ✓ Meaningful user tasks performed.
- ✓ Maintain consistency across a family of applications.
- ✓ Keep interaction results the same as per user expectations.

